

THE CLAUDE BEAST

/greptile-review-loop

Autonomous Review-Fix-Merge for Every Pull Request

Zero Human Intervention | Max 5 Iterations | Auto-Merge at 4/5

What Is /greptile-review-loop?

A fully autonomous agent that manages the entire PR lifecycle

Core Concept

One command kicks off a closed loop: detect/create PR → trigger Greptile review → analyze comments → fix issues in parallel → push → re-review → repeat until merge or hard stop.

Capabilities

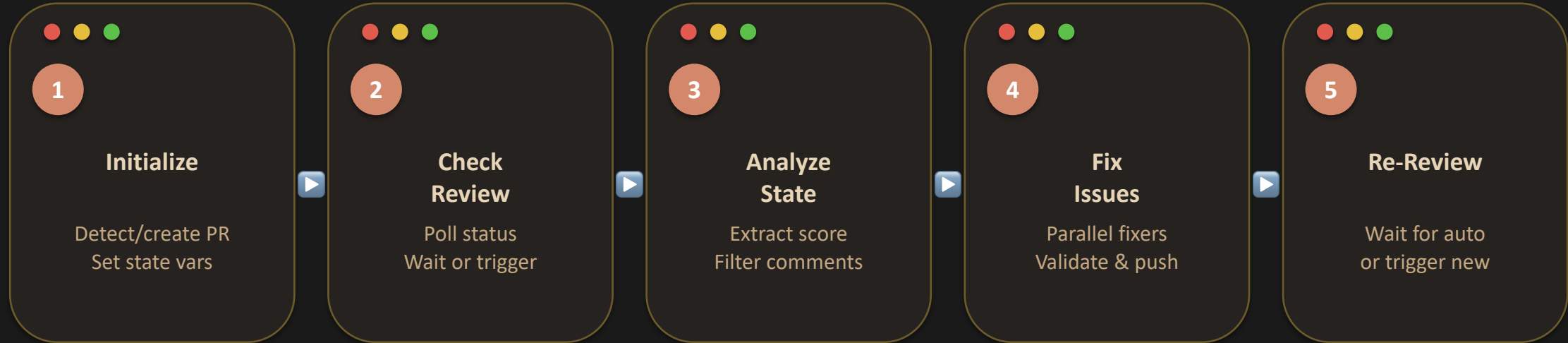
- Auto-detects or creates PR from current branch
- Triggers Greptile code reviews via @greptileai
- Spawns parallel code-fixer agents (one per file)
- Validates fixes with tsc, py_compile, go build, cargo
- Squash-merges at score $\geq 4/5$ + zero unaddressed
- Outputs final status report with iteration count

Safety Guarantees

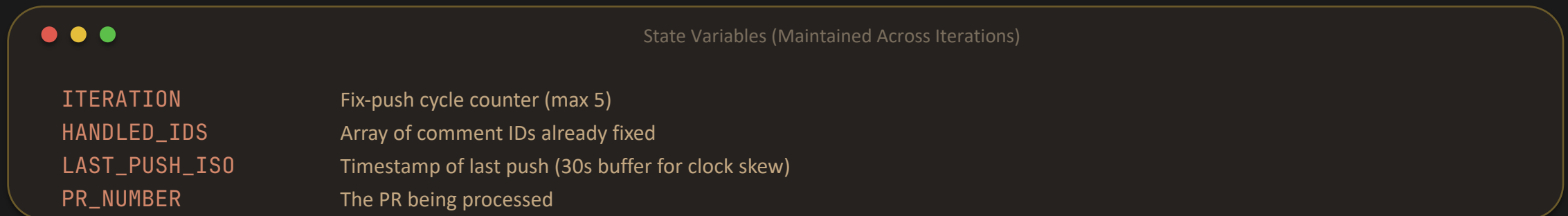
- NEVER re-fixes previously handled comments
- NEVER triggers review while one is in progress
- NEVER merges PR with score $\leq 3/5$
- NEVER loops beyond 5 iterations
- NEVER pushes code without validation first
- Tracks HANDLED_IDS to prevent duplicate work

The 5-Step Loop Architecture

Cyclic execution with state tracking across iterations



🔄 Steps 2-5 repeat until: score $\geq 4/5$ AND unaddressed = 0 OR hard stop triggered



Parallel Code-Fixer Agents

One agent per affected file -- parallel execution for maximum speed

How Code-Fixer Works

- Issues grouped by file path
- Task tool spawns one agent per file
- Each agent reads file + Greptile comments
- Agent applies targeted fix
- Language-specific validation runs
- Only modified files are staged for commit

Validation by Language

TypeScript

`tsc --noEmit`

Python

`py_compile`

Go

`go build ./...`

Rust

`cargo check`



Agent Isolation Properties

- Single-file scope -- each agent can only modify its assigned file, preventing cascading changes
- Strict verification -- fixes must pass compilation before being committed
- Parallel execution -- 5 file issues spawn 5 agents simultaneously, not sequentially

Greptile MCP Integration

Six MCP tools orchestrated in a precise sequence



`get_merge_request()`

Primary source of truth

Addressed vs unaddressed
comments, PR analysis



`list_code_reviews()`

Review status tracking

PENDING, REVIEWING,
COMPLETED, FAILED



`get_code_review()`

Score extraction

Parses X/5 pattern
from review body



`list_merge_request_comments()`

Comment filtering

greptileGenerated=true
addressed=false



`@greptileai comment`

Review triggering

Via gh pr comment
(not MCP trigger tool)



`Greptile auto-review`

Change detection

2-min wait for auto
before manual trigger



Key Design Decision

Reviews are triggered via GitHub comment (`@greptileai please review`), NOT the MCP `trigger_code_review` tool. This respects Greptile's auto-review detection and avoids duplicate reviews.

Hard Stops & Safety Mechanisms

HARD STOP CONDITIONS

Successful merge Score $\geq 4/5$ + 0 unaddressed

Score too low Score $\leq 3/5$ after review

Max iterations 5 fix-push cycles completed

Build failure Validation (tsc, go, etc.) fails

Review timeout No review in 10 minutes

No score found Cannot extract X/5 pattern

SAFETY MECHANISMS

   **Duplicate Prevention**
HANDLED_IDS tracks every fixed comment -- never re-processes

   **Timestamp Filtering**
LAST_PUSH_ISO + 30s buffer ensures only post-push comments

   **Single-File Isolation**
Each code-fixer can only modify its assigned file

   **Auto-Review Awareness**
2-min wait before manual trigger avoids duplicates

   **Validation Gate**
All fixes must pass compilation before commit



Design principle: fail safe, fail fast. Every decision point has an explicit exit condition. The agent never enters an unbounded loop or pushes unvalidated code.

Installation & Developer Workflow

Prerequisites

- Claude Code CLI + GitHub CLI (gh)
- Greptile GitHub App installed on repo
- Greptile MCP Plugin active in Claude Code

```
git clone  
https://github.com/startappdev/agents.git  
cd agents && ./install.sh
```

Required Components

greptile-review-loop.md

Agent definition

~/.claude/agents/

code-fixer.md

Sub-agent definition

~/.claude/agents/

Typical Workflow

- 1 Write code on feature branch
- 2 Type /greptile-review-loop
- 3 Agent creates/finds PR
- 4 Greptile reviews → agent fixes
- 5 Loop until 4/5 → auto-merge
- 6 Read final status report

greptile-review-loop/

Slash command

~/.claude/commands/

post-pr-greptile-hook.sh

Post-PR notification

~/.claude/hooks/

Final Status Report & The Vision

Example Final Report

PR: #42 (feature/auth-refactor)

Final Score: 5/5

Iterations: 3

Status: MERGED

Fixes Applied:

- src/auth/validate.ts: input sanitization
- src/api/users.ts: parameterized queries
- tests/auth.test.ts: added edge cases

Total Time: ~12 minutes

The Autonomous Development Vision

1

Human Express Intent

Developer writes code and pushes

2

Coding Agent Executes

Claude Code builds the feature

3

Validation Agent Reviews

Greptile analyzes with full context

4

Fix Agent Remediates

Code-fixer addresses findings

5

Loop Until Approved

Repeat until quality threshold met

The Key Insight

/greptile-review-loop is not just a tool -- it's a proof of concept for the autonomous development loop. The human writes code; agents review, fix, and merge. The developer's role shifts from code reviewer to intent expresser.

Why `/greptile-review-loop` Matters

Truly Autonomous

One command. Zero human intervention. PR created, reviewed, fixed, and merged automatically.

Safety by Design

5-iteration cap, HANDLED_IDS deduplication, build validation, score thresholds -- every edge case has an explicit exit.

The Future of Code Quality

Human writes code. Agent reviews. Agent fixes. Agent merges. The review bottleneck is eliminated.

github.com/startappdev/agents | MIT License | `/greptile-review-loop`